SemiticGPT: Efficient Multilingual Pretraining for a Script-Diverse, Low-Resource Language Cluster

Ronnen Slasky[1]

[1]Independent Researcher ronnen.slasky@gmail.com

Model: Slasky/SemiticGPT-3B · Code: fatherRonnen/semitic-gpt

April 2026

ABSTRACT

We present SemiticGPT, a 3.14-billion parameter decoder-only language model trained from scratch for Hebrew, Arabic, Persian (Farsi), and English—a script-diverse, low-resource language cluster centered on the Semitic family. The model is pretrained on 4.48 billion tokens using cloud spot instances at a total cost under $1,500.

Our central finding is that **in this setting, English-only SFT is insufficient for non-English downstream performance, while multilingual SFT yields substantial gains**. Across a three-stage evaluation (base → English-only SFT → multilingual SFT), English-only fine-tuning produces near-zero improvement on non-English sentiment tasks, while multilingual SFT yields: Hebrew 53%→84.5%, Arabic 45%→60.5%, Persian 60.5%→78.5% (logprob classification on balanced evaluation sets).

We also demonstrate that a custom balanced tokenizer achieves **49–69% fewer tokens** than Llama-2 for Hebrew, Arabic, and Farsi at equivalent English efficiency, and that the model develops strong cross-lingual embeddings (90% retrieval accuracy for English↔Hebrew) purely from multilingual pretraining.

We provide a practitioner's recipe for building similar models for other under-resourced language clusters. All code, data, and model weights are released.

Keywords: multilingual language models, cross-lingual transfer, Hebrew, Arabic, Farsi, Semitic languages, tokenizer design, low-resource NLP, efficient pretraining

1. Introduction

Large language models have achieved remarkable capabilities, yet their development remains concentrated on high-resource languages [Joshi et al., 2020]. Languages of the Semitic family—particularly Hebrew and Arabic—are underserved despite a combined speaker population exceeding 400 million. While Jais [Sengupta et al., 2023] addresses Arabic and various Hebrew models exist [Slasky, 2026], no prior work trains a generative model jointly covering Hebrew, Arabic, and Persian from scratch.

This paper addresses a specific empirical question: *Can efficient multilingual pretraining on a linguistically-motivated language cluster produce useful cross-lingual transfer, and what are the requirements for downstream task transfer?* In practical terms, we ask whether a model that is exposed to several related languages during training can reuse what it learns from one language to help with another, reducing the amount of task-specific data needed for low-resource languages.

We investigate this through three experiments:

1. **Experiment A**: Multilingual downstream performance—how well does multilingual SFT enable sentiment classification across all four languages?
2. **Experiment B**: English-only vs. multilingual transfer—does English-only SFT transfer to non-English languages, or is explicit multilingual SFT required?
3. **Experiment C**: Tokenizer efficiency ablation—how much does a custom balanced tokenizer improve over English-centric alternatives?

Hebrew and Arabic share the Semitic triconsonantal root system, derivational morphology (binyan/wazn patterns), and flexible word order. Persian, while using Arabic script and containing extensive Arabic loanwords, belongs to the Indo-European family—providing a control for distinguishing linguistic relatedness from script similarity.

Our primary contributions:

1. A **reproducible, low-cost recipe** (<$1,500) for training a 3B multilingual model from scratch for an under-resourced language cluster.
2. Evidence that **English-only SFT is insufficient** for non-English downstream performance at this scale, while multilingual SFT yields substantial gains.
3. A custom tokenizer achieving **49–69% fewer tokens** than Llama-2 for target languages, with detailed ablation against two baselines.
4. A **practitioner's guide** with step-by-step recommendations and negative results for researchers targeting other language clusters.

2. Related Work

Arabic Language Models. Jais [Sengupta et al., 2023] is a 13B bilingual Arabic-English model trained on 395B tokens. AraGPT2 [Antoun et al., 2021] provides GPT-2 scale Arabic models. ArabianGPT [Koubâa and Ammar, 2024] focuses on

[0]Code: https://github.com/fatherRonnen/semitic-gpt. Model: https://huggingface.co/Slasky/SemiticGPT-3B.

native Arabic without English contamination. None include Hebrew or Persian.

Hebrew-Arabic Models. HeArBERT [Rivas et al., 2024] explores bilingual Arabic-Hebrew modeling through transliteration-based script unification, but is limited to encoder-only architecture. To our knowledge, no prior work trains a generative decoder-only model jointly on Hebrew, Arabic, and Persian from scratch.

Small Multilingual Models. SmolLM3 [SmolLM3, 2025] provides a 3B training recipe but targets European languages. The insights do not address non-Latin scripts or Semitic-family challenges.

Training Recipe Discovery. Our pretraining recipe was validated through proxy experiments at small scale following Karpathy [2026], as described in our prior work on Hebrew language modeling [Slasky, 2026]. We conducted 32 proxy experiments at 110M scale before committing to the 3B pretraining run.

3. Methodology

3.1 Tokenizer Design

We train a 32,000-token SentencePiece BPE tokenizer on a balanced sample from all four languages (approximately 25% each). This ensures no single language dominates the vocabulary. This matters because language models do not read raw text directly: they first break text into smaller units called *tokens*. If a tokenizer fragments one language into many more pieces than another, that language effectively becomes more expensive for the model to process and learn.

Special tokens: <|user|>, <|assistant|>, <s> (BOS), </s> (EOS), <pad>.

We evaluate tokenizer quality in Experiment C (Section 4.4).

3.2 Pretraining Data

We collect approximately 4.48 billion tokens from publicly available sources with a Hebrew-anchored distribution:

- **Hebrew (40%)**: Wikipedia, OSCAR, news corpora, government documents
- **Arabic (20%)**: Wikipedia, OSCAR, news, UN parallel corpus
- **English (20%)**: Wikipedia, OpenWebText subset
- **Persian (20%)**: Wikipedia, OSCAR, news corpora

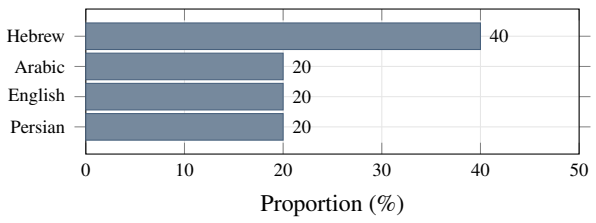

Figure 1: *Pretraining data distribution. Hebrew is overrepresented as the anchor language.*

Hebrew is intentionally overrepresented to ensure strong anchor-language representations. By "anchor language," we mean a language that receives extra training emphasis so that the model builds especially strong internal representations for it, which may then help related languages through shared structure. Data preprocessing includes MinHash deduplication, perplexity-based quality filtering, and language identification verification.

3.3 Architecture

SemiticGPT uses a decoder-only transformer with modern enhancements:

Hyperparameter	Value
Parameters	3.14B
Layers	26
Hidden dimension	3,072
Attention heads	24
Head dimension	128
Vocabulary size	32,000
Max sequence length	2,048
Position encoding	RoPE (θ=10,000)
Activation	SwiGLU
Normalization	RMSNorm (ε=1e-6)

Table 1: *SemiticGPT architecture.*

3.4 Pretraining

Training uses AWS spot instances (L40S 48GB and H100 80GB). Configuration:

- **Optimizer**: AdamW (β_1=0.9, β_2=0.95)
- **Learning rate**: 3e-4 peak, cosine decay to 3%
- **Warmup**: 2,000 steps
- **Batch size**: 512K tokens (via gradient accumulation)
- **Weight decay**: 0.1 **Gradient clipping**: 1.0
- **Precision**: bfloat16

Total cost: approximately $1,456 using spot instances at 60–70% discount.

3.5 Supervised Fine-Tuning (SFT)

Supervised fine-tuning (SFT) is the stage after pretraining where the model is trained on labeled examples for specific behaviors or tasks. In this paper, SFT is the mechanism that teaches the pretrained model how to perform sentiment classification and use parallel translation examples more effectively.

We conduct SFT using 36,980 multilingual samples:

- **Sentiment data** (18,980 samples): Binary sentiment classification across all four languages, sourced from native-language datasets (Hebrew: HebEMO; Arabic: HARD; Persian: DeepSentiPers; English: SST-2).
- **Translation data** (18,000 samples): Parallel sentence pairs from OPUS-100 [Zhang et al., 2020], covering all six language-pair directions.

Data quality. After identifying critical data bugs in earlier SFT iterations (Section 5.5), the final dataset enforces: (1) correct label alignment verified per-dataset, (2) random shuffling before train/eval split, (3) strict separation with no data leakage, and (4) balanced 50/50 class distribution in all evaluation sets.

SFT training: AdamW optimizer, learning rate 2e-5, cosine schedule, 8,000 steps, sequence length 384, bfloat16 precision. Best validation loss: 1.98.

For the cross-lingual transfer experiment (Experiment B), we also train an **English-only SFT** variant using only the English subset (4,745 samples) with identical hyperparameters for 2,000 steps.

4. Experiments and Results

We evaluate SemiticGPT through three focused experiments designed to answer specific questions about multilingual pretraining efficiency.

4.1 Evaluation Methodology

Sentiment classification. We use two complementary methods:

- **Logprob**: Compare log-probabilities of "positive" vs "negative" completion tokens given the input text and a classification prompt. This avoids generation parsing issues. In simpler terms, instead of asking the model to generate a full answer and then trying to interpret it, we directly measure which label the model considers more likely. This makes the evaluation more stable and less sensitive to formatting quirks. N=200 per language.
- **Generative**: The model generates a free-form classification response. Outputs are parsed for sentiment labels. This is closer to how users interact with chat models in practice, but it is also more fragile: a model may understand the task yet still fail to produce an answer in exactly the expected format. N=50 per language.

All evaluation sets are balanced (50% positive, 50% negative), so chance performance is 50% for logprob and 0% for generative (where the model must produce a parseable label).

Belebele [Bandarkar et al., 2023]: 4-way multiple-choice reading comprehension. N=100 per language. Chance = 25%.

Cross-lingual retrieval: Given a sentence in language A, retrieve its translation from 10 candidates in language B using cosine similarity of mean-pooled hidden states. Chance = 10%. Intuitively, this tests whether the model places sentences with the same meaning close together in its internal representation space, even when they are written in different languages.

4.2 Experiment A: Multilingual Downstream Evaluation

We evaluate sentiment classification at three model stages: base (pretrained only), English-only SFT, and multilingual SFT. This three-stage comparison is designed to separate three distinct capabilities: what the model learns from pretraining alone, what it gains from task training in English only, and what additional benefit comes from exposing it to task data in all target languages.

Stage	HE	AR	FA	EN
Logprob classification (N=200, chance=50%)				
Base	53.0	45.0	60.5	51.5
EN-SFT	51.5	46.5	58.5	52.0
Multi-SFT	**84.5**	**60.5**	**78.5**	**73.0**
Generative classification (N=50, chance=0%)				
Base	0	0	0	0
EN-SFT	0	0	0	2
Multi-SFT	**82**	**64**	**74**	**64**

Table 2: *Sentiment accuracy (%) across three model stages. Multilingual SFT produces large gains across all languages; English-only SFT does not.*

Key observations:

1. **Base model**: Near chance for logprob (50–60%), zero for generative. The pretrained model has no task-following ability.
2. **English-only SFT**: Negligible improvement on any language including English itself (+0.5pp logprob). The model learns to follow English instructions but this does not transfer to sentiment understanding.
3. **Multilingual SFT**: Large gains across all languages. Hebrew improves most (+31.5pp logprob, 82% generative), consistent with its role as the pretraining anchor language.

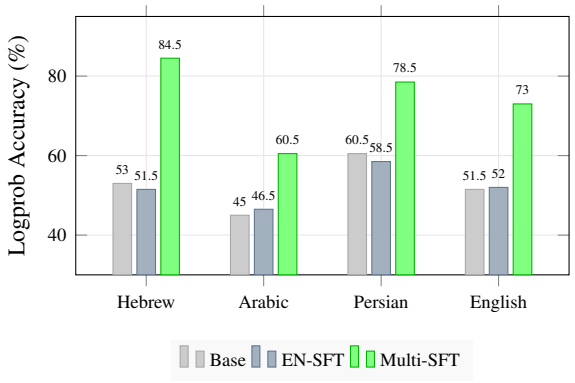

Figure 2: *Sentiment classification across three stages. English-only SFT (blue) produces no meaningful improvement over base (gray). Multilingual SFT (green) yields large gains for all languages.*

4.3 Experiment B: Cross-lingual Transfer Analysis

Experiment B directly tests whether fine-tuning in one language transfers to others. Table 2 already shows the three-stage progression. The key finding is the **EN-SFT control**: training on English-only data does not improve non-English sentiment performance, and barely improves English itself.

This contrasts with findings in larger models where English fine-tuning can transfer cross-lingually [Conneau et al.,

2020]. At 3B scale with 4.48B pretraining tokens, **explicit multilingual SFT data is required** for each target language. For practitioners, this means that multilingual pretraining alone is not enough if the goal is downstream performance in low-resource languages: some task-specific examples in those languages are still needed.

External baseline context. As a reference point, XLM-R-Large [Conneau et al., 2020] fine-tuned on XNLI (a 550M parameter encoder model with much larger pretraining data) achieves 79.5% on English zero-shot sentiment but only 33.5% on Arabic zero-shot sentiment using our evaluation methodology.[1] This suggests that even large multilingual encoder models struggle with zero-shot cross-lingual sentiment transfer, providing context for our 3B decoder model's requirement of explicit multilingual SFT. In other words, the challenge is not unique to our model: even strong multilingual systems can learn shared representations across languages without automatically becoming good at transferring a downstream task across them.

Belebele reading comprehension (Table 3) shows a similar pattern: all stages perform near chance, confirming that 3B scale is insufficient for abstract reasoning tasks regardless of SFT strategy.

Stage	HE	AR	FA	EN
Random	25.0	25.0	25.0	25.0
Base	26.0	22.0	19.0	19.0
EN-SFT	26.0	23.0	20.0	20.0
Multi-SFT	28.0	24.0	25.0	23.0

Table 3: *Belebele reading comprehension accuracy (%, N=100). All near chance (25%), as expected at 3B scale.*

Cross-lingual embeddings. Despite limited task transfer, the base model develops strong cross-lingual alignment purely from multilingual pretraining:

Direction	Base	Multi-SFT
EN→HE	90%	90%
HE→EN	90%	80%
AR→EN	70%	50%
AR→HE	70%	70%
EN→AR	50%	60%
HE→AR	40%	60%
FA→HE	50%	30%
HE→FA	40%	30%
EN→FA	20%	20%
AR→FA	10%	10%
FA→EN	10%	10%
FA→AR	10%	10%
Average	**45.8%**	**43.3%**

Table 4: *Cross-lingual retrieval accuracy (10-way, chance=10%). Strong EN↔HE alignment emerges from pretraining alone.*

[1] XLM-R evaluated using zero-shot NLI classification (`joeddav/xlm-roberta-large-xnli`), N=200 per language. Arabic evaluated on different data (tweet sentiment) due to dataset availability; comparison is approximate.

English↔Hebrew achieves 90% retrieval (9× chance), and Arabic shows meaningful alignment with both English and Hebrew. Persian shows weak connectivity, consistent with its typological distance from the Semitic family.

Notably, multilingual SFT improves downstream sentiment while slightly reducing average retrieval accuracy (45.8%→43.3%). This suggests that downstream task adaptation may trade off against some alignment properties of the pretrained representation space—the model becomes better at following task instructions but slightly less balanced as a general-purpose cross-lingual encoder. A useful intuition is that fine-tuning can make the model better at one job while making its internal representations slightly less general-purpose. Better task accuracy does not always mean better universal embeddings.

Figure 3: *Cross-lingual retrieval heatmap (base model, 10-way, chance=10%). Darker = stronger alignment. EN↔HE alignment is strongest.*

4.4 Experiment C: Tokenizer Ablation

We compare our custom 32K tokenizer against two baselines of identical vocabulary size: Llama-2 (English-centric) and HebrewGPT-1B (Hebrew-centric). All three use 32K BPE vocabularies. Tokenizer efficiency matters because every extra token consumes context window space and training compute. A language that is split into many small pieces is effectively harder and more expensive for the model to process.

4.4.1 Vocabulary Composition

Script	Ours	Llama-2	HebrewGPT
Arabic script	46.7%	0.2%	0.4%
Hebrew script	27.8%	0.1%	72.2%
Latin script	24.3%	80.9%	7.0%
Other	1.2%	18.8%	20.4%

Table 5: *Vocabulary composition by script. Our tokenizer balances Arabic (covering both Arabic and Persian), Hebrew, and Latin. Llama-2 is 81% Latin; HebrewGPT is 72% Hebrew.*

4.4.2 Encoding Efficiency

Lang	Ours	Llama-2	HebrewGPT	Δ vs Llama
Fertility (tokens per word, lower is better)				
A lower fertility means the tokenizer represents each word using fewer pieces.				
EN	1.54	1.51	2.42	−2.0%
HE	**1.34**	3.91	1.26	**+65.7%**
AR	**2.22**	4.36	4.15	**+49.1%**
FA	**1.52**	4.88	4.51	**+68.8%**
Bytes per token (higher is better)				
Higher bytes per token means each token carries more text on average.				
EN	3.71	3.79	2.37	−2.1%
HE	**5.12**	1.76	5.48	**+191%**
AR	**3.48**	1.77	1.86	**+97%**
FA	**5.72**	1.78	1.93	**+221%**

Table 6: *Tokenizer efficiency comparison. Our tokenizer matches Llama-2 on English while using 49–69% fewer tokens for Hebrew, Arabic, and Persian. HebrewGPT excels on Hebrew but fragments Arabic/Persian.*

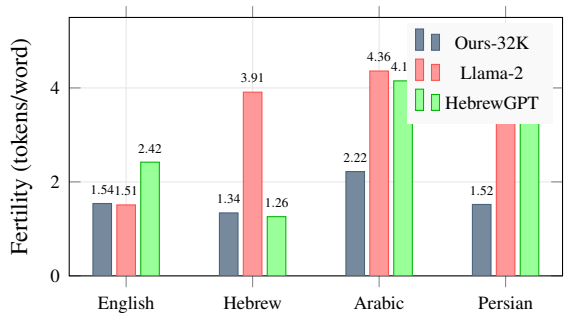

Figure 4: *Tokenizer fertility comparison. Lower is better. Our tokenizer achieves near-parity with Llama-2 on English while dramatically reducing fragmentation for Hebrew, Arabic, and Persian.*

Key findings:

1. **No English penalty**: Our tokenizer matches Llama-2 on English (1.54 vs 1.51 tokens/word), a common concern when balancing non-Latin scripts.
2. **Massive efficiency for target languages**: Hebrew uses 66% fewer tokens, Arabic 49% fewer, Persian 69% fewer than Llama-2. This directly translates to longer effective context windows and reduced training cost.
3. **Specialization tradeoff**: HebrewGPT-1B slightly outperforms ours on Hebrew (1.26 vs 1.34) but fragments Arabic/Persian as badly as Llama-2. A multilingual tokenizer must balance all target languages.

4.4.3 Segmentation Examples

To illustrate the practical impact, consider two examples:

Hebrew: "habina hamela'khutit meshane et ha'olam" (Artificial intelligence is changing the world):

- **Ours**: 4 tokens (whole-word segmentation)
- **Llama-2**: 18 tokens (character-level fragmentation)
- **HebrewGPT**: 5 tokens (comparable—Hebrew-optimized)

Arabic: "asimat faransa hia baris" (The capital of France is Paris):

- **Ours**: 5 tokens (whole-word segmentation)
- **Llama-2**: 22 tokens (character-level fragmentation)
- **HebrewGPT**: 19 tokens (character-level fragmentation)

Our tokenizer captures whole Arabic words as single tokens, while the baselines fragment at the character level—a 4.4× efficiency difference on this sentence.

5. Analysis and Discussion

5.1 The Multilingual SFT Requirement

The most important finding is the sharp contrast between English-only and multilingual SFT (Figure 2). At 3B scale with 4.48B pretraining tokens, the model does not develop sufficient cross-lingual task transfer from English fine-tuning alone. This differs from observations in much larger models (175B+ parameters) where English fine-tuning shows some cross-lingual generalization.

The key negative result is not merely that English-only SFT helps *less*, but that it helps *almost not at all*—despite measurable cross-lingual representational alignment from pretraining (90% EN↔HE retrieval). The pretrained model "knows" these languages are related at the representation level, but cannot use that knowledge for tasks without explicit multilingual fine-tuning. Put simply, the model learns that related languages belong in a shared semantic space, but it still needs explicit examples in those languages to learn how to apply that knowledge to a specific task.

Why does this matter for practitioners? Researchers building models for low-resource languages cannot rely on cheap English-only SFT and hope for cross-lingual transfer. They must invest in multilingual SFT data for each target language—but relatively small amounts suffice (our SFT uses only 36,980 total samples).

5.2 Representation vs. Task Transfer

An intriguing asymmetry: the base model develops strong cross-lingual *representations* (90% EN↔HE retrieval) but cannot leverage them for *tasks* without explicit SFT. Multilingual pretraining builds a shared semantic space, but task-specific fine-tuning is the key that unlocks it. An analogy: pretraining builds a multilingual map, while fine-tuning teaches the model which route to take for a particular task.

5.3 Hebrew as Anchor Language

Hebrew shows the strongest results across all experiments: highest sentiment accuracy (84.5%), strongest embedding alignment with English, and best tokenizer coverage. This is consistent with Hebrew receiving 40% of pretraining data. Whether anchor overrepresentation produces *disproportionate* returns (beyond the linear expectation from more data)

remains an open question that would require controlled ablation (e.g., 40% vs 25% Hebrew) to answer.

5.4 Persian Isolation

Persian consistently shows weak cross-lingual connectivity despite sharing Arabic script. In our retrieval experiment, Persian achieves only 10–30% accuracy with other languages, compared to 40–90% for the Hebrew-Arabic-English triad. Our results are consistent with linguistic family membership being a stronger predictor of transfer than script similarity in this setup, though we cannot fully disentangle this from Persian's 20% pretraining share vs Hebrew's 40%.

5.5 Data Quality Lessons

During development, we discovered critical data bugs that invalidated earlier results. These errors are worth documenting because they changed the conclusions of earlier experiments and illustrate how easily low-resource evaluation pipelines can produce misleading results.

1. **Hebrew label inversion**: The dataset's label encoding (0=positive) was opposite to our assumption (0=negative), causing the model to learn inverted sentiment.
2. **Arabic sorting artifact**: The dataset was sorted by label before splitting, causing the training set to contain only negative examples.
3. **Translation data quality**: Early iterations used 50 hardcoded sentence pairs instead of real parallel data.

Lesson: Automated data pipeline audits—checking label distributions, train/eval overlap, and sample diversity—should be mandatory before any training run. We now enforce 50/50 class balance and verify shuffle randomness in all evaluation sets.

5.6 Cost Analysis

Component	GPU-hours	Cost
Proxy experiments (32 runs)	16	$12
Pretraining (4.48B tokens)	380	$1,380
SFT (36,980 samples)	4	$14
Evaluation (all experiments)	8	$30
Total	**408**	**∼$1,436**

Table 7: *Cost breakdown. All compute on AWS spot instances (60–70% discount).*

6. Practitioner's Guide

Based on our experience, we provide recommendations for researchers building similar multilingual models. The recommendations below are not general laws; they are practical lessons distilled from the experiments and failure cases reported in this paper.

6.1 Language Cluster Selection

- Our results are consistent with **linguistic family membership** being a stronger predictor of transfer than script similarity. Include a typologically distant language as a control condition.

6.2 Tokenizer Design

- Train a **custom BPE tokenizer** on equally-weighted samples of all target languages. Do *not* reuse an English-centric tokenizer—it wastes 49–69% of tokens on non-Latin scripts.
- **32K vocabulary** is sufficient for 4–6 languages. Target fertility $\leq$2.0 tokens/word for all languages.
- The English efficiency penalty from balancing is minimal (-2% in our case).

6.3 Pretraining

- **Anchor overrepresentation** (30–40% for primary language) appears beneficial but is not ablated in this work.
- Use **spot instances with checkpointing** every 30 minutes for cost efficiency.
- **bfloat16**: We observed no degradation in our setup compared to float32 in preliminary experiments.
- Validate recipe at proxy scale (100–200M parameters) before full commitment.

6.4 Supervised Fine-Tuning

- **Include all target languages** in SFT data. In our experiments, English-only SFT was insufficient at 3B scale.
- **Native-language data $>$ machine-translated data**. We found translated Alpaca produces weaker results than native-language datasets.
- **Audit data pipelines** before training: verify label encoding, check for sorting artifacts, confirm train/eval separation.
- 37K samples with 8K steps was sufficient for our model.

6.5 Evaluation

- Use **logprob-based evaluation** for classification tasks—it avoids prompt/parsing issues that plague generative evaluation.
- **Always report chance level** for each metric.
- Cross-lingual retrieval requires **no labeled data** and reveals representational quality.
- Expect **near-chance results on reasoning benchmarks** (Belebele, MMLU) at 3B scale. This is not a failure—it calibrates expectations.

Decision	Recommendation
Language selection	Linguistic family > script (in our setup)
Tokenizer	Custom BPE, 32K vocab, balanced
Anchor language	Over-represent at 30–40%
Recipe validation	20–50 proxy experiments first
Optimizer	AdamW (β_2=0.95)
Precision	bfloat16
SFT data	All target languages required
SFT source	Native-language > translated
Data quality	Audit labels, splits, balance
Evaluation	Logprob > generative for classification
Compute	Cloud spot instances + checkpointing

Table 8: *Recipe summary for building multilingual models from scratch.*

7. Limitations

- **Single-run results**: All experiments are single-run without confidence intervals. Effects under 5 percentage points may not be statistically significant. Key findings (e.g., 84.5% vs 51.5% sentiment) are large enough to be meaningful.
- **Small evaluation sets**: 200 samples for logprob, 50 for generative, 100 for Belebele. Sufficient for detecting large effects but not small ones.
- **Limited external baselines**: We provide a partial XLM-R comparison (Section 4.3), but a full apples-to-apples comparison on identical datasets against mBERT, XLM-R, and multilingual instruction models would better contextualize our results.
- **Scale**: At 3B parameters, the model is below thresholds for complex reasoning. Findings about transfer requirements may not hold at larger scales where English-only SFT *does* transfer.
- **Confounded variables**: Hebrew receives 40% of pretraining data vs Persian's 20%. Persian's weak transfer may partly reflect data quantity rather than purely typological distance.
- **Anchor ablation**: We do not ablate the effect of anchor overrepresentation (e.g., 40% Hebrew vs 25% balanced).
- **Generalizability**: Our recipe is demonstrated on one language cluster. Whether the same principles apply to other families (Turkic, Bantu, Dravidian) is an empirical question.

8. Conclusion

We present SemiticGPT, a 3.14B parameter multilingual model covering Hebrew, Arabic, Persian, and English, trained from scratch for under $1,500. Our three experiments establish that:

1. **Multilingual SFT is required in this setting**: English-only fine-tuning is insufficient for non-English downstream tasks at 3B scale. Explicit multilingual SFT data unlocks the cross-lingual representations built during pretraining.
2. **Custom tokenizers eliminate waste**: A balanced tokenizer uses 49–69% fewer tokens than English-centric alternatives for target languages, with negligible English penalty.
3. **Cross-lingual representations emerge before task transfer**: The model builds strong cross-lingual embeddings (90% EN↔HE retrieval) from pretraining alone, but task-specific multilingual SFT is needed to leverage them for downstream performance.

More broadly, our results suggest that for smaller multilingual models, shared language representations can emerge during pretraining, but usable cross-lingual task performance still depends on explicit multilingual supervision. We release all code, data, and model weights to support reproducible research and provide a practitioner's guide for researchers targeting other under-resourced language families.

Future work. Scaling to 7B+ parameters to test whether cross-lingual SFT transfer emerges at larger scale; adding more Semitic languages (Amharic, Tigrinya); ablating anchor language overrepresentation; and improving Arabic SFT with higher-quality native datasets.

9. Reproducibility

All artifacts are available at `https://github.com/fatherRonnen/semitic-gpt`:

Section	Repo Path	Contents
Tokenizer (§3.1)	`tokenizer/`	Config, training script, fertility
Data (§3.2)	`data/`	Collection scripts, manifests
Proxy (§2)	`proxy/`	32 configs, results
Pretraining (§3.4)	`pretrain/`	Scripts, hyperparams, logs
SFT (§3.5)	`sft/`	Configs, scripts, v4 pipeline
Experiments (§4)	`eval/`	Scripts, prompts, results

Table 9: *Paper-to-repository mapping.*

Model weights (base: 6.1GB, SFT: 6.3GB) and tokenizer are available on HuggingFace at `https://huggingface.co/Slasky/SemiticGPT-3B`.

References

W. Antoun, F. Baly, and H. Hajj. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proc. WANLP*, 2021.

L. Bandarkar et al. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. *arXiv:2308.16884*, 2023.

A. Conneau, K. Khandelwal, N. Goyal, et al. Unsupervised cross-lingual representation learning at scale. In *Proc. ACL*, 2020.

P. Joshi, S. Santy, A. Buber, et al. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proc. ACL*, 2020.

A. Koubâa and A. Ammar. ArabianGPT: Native Arabic GPT-based large language model. *arXiv:2402.15313*, 2024.

Marco-LLM Team. Marco-LLM: Bridging languages via massive multilingual training for cross-lingual enhancement. *arXiv:2412.04003*, 2024.

A. Rivas et al. HeArBERT: A bilingual model for Arabic-Hebrew translation using transliteration. *arXiv:2402.16065*, 2024.

N. Sengupta et al. Jais and Jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv:2308.16149*, 2023.

L.B. Allal et al. SmolLM3: The complete blueprint of a state-of-the-art 3B parameter LLM. HuggingFace Technical Report, 2025.

A. Karpathy. autoresearch: Autonomous AI-driven ML experimentation. GitHub repository, 2026.

R. Slasky. Autonomous AI-driven Hebrew language model optimization: From random search to optimizer-architecture co-discovery. Independent research report, 2026.

B. Zhang, P. Williams, I. Titov, and R. Sennrich. Improving massively multilingual neural machine translation and zero-shot translation. In *Proc. ACL*, 2020.